Région Hauts-de-France
Cofinancé par l'Union européenne
realconseil DIAGNOSTIC ACCOMPAGNEMENT FORMATION

TP - Mini blog PHP avec PDO (initiation CRUD)

Objectif pédagogique

Créer un mini blog très simple en PHP permettant de manipuler une base de données via **PDO**.

L'objectif principal n'est **pas le design** ni la sécurité avancée, mais :

- comprendre le lien entre **PHP et SQL**
- manipuler les données avec **PDO**
- comprendre le fonctionnement de **GET / POST**
- apprendre à **chercher de l'aide** (documentation, IA)
- être capable de se **débrouiller face à un problème**

À la fin du TP, le blog devra permettre :

- d'afficher une liste d'articles
- d'ajouter un article
- de supprimer un article

Contraintes techniques

- PHP (procédural autorisé)
- PDO obligatoire
- Requêtes préparées (`prepare()` + `execute()`)
- Pas de framework
- HTML simple (pas de Bootstrap, pas d'icônes)
- Code lisible et commenté

Base de données

Importer la base suivante dans MariaDB :

```sql
CREATE DATABASE blog CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE blog;

CREATE TABLE post (
  post_id INT AUTO_INCREMENT PRIMARY KEY,
  post_title VARCHAR(255) NOT NULL,
  post_excerpt TEXT,
  post_content TEXT NOT NULL,
  post_author VARCHAR(100) NOT NULL,
```

```
  post_created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  post_updated_at DATETIME DEFAULT NULL,
  post_is_published TINYINT(1) NOT NULL DEFAULT 1
);

INSERT INTO post (post_title, post_excerpt, post_content, post_author)
VALUES
(
  'Bienvenue sur le blog',
  'Premier article de démonstration',
  'Ceci est le contenu du premier article. Il sert à tester le CRUD en PHP
avec PDO.',
  'Admin'
),
(
  'PDO et sécurité',
  'Pourquoi utiliser des requêtes préparées',
  'Les requêtes préparées permettent de se protéger contre certaines
injections SQL...',
  'Manu'
);
```

Structure du projet (recommandée)

```
blog/
  db.php
  index.php
  create.php
  delete.php
  README.md
  docs/
    IA.md
```

En cas de difficultés à découper le code Php se réferer au chapitre PHP_09_Les_bibliotheques_de_fonctions.html du cours. Demandez à votre formateur en cas de pépin.

Connexion à la base de données (db.php)

La connexion à la base doit être centralisée dans un seul fichier.

Contraintes :

- charset `utf8mb4`
- `ERRMODE_EXCEPTION`
- `FETCH_ASSOC`

Exemple :

```php
<?php
$dsn = 'mysql:host=localhost;dbname=blog;charset=utf8mb4';
$user = 'admin';
$pass = 'motdepasse';

$options = [
  PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
  PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
];

$pdo = new PDO($dsn, $user, $pass, $options);
```

Pages à réaliser

1. index.php - Liste des articles

Cette page affiche la liste de tous les articles du plus récent au plus ancien.

Pour chaque article, afficher :

- le titre
- l'auteur
- la date de création
- un lien **Supprimer**

Un lien **Ajouter un article** doit être présent.

Requête SQL attendue :

```sql
SELECT post_id, post_title, post_author, post_created_at
FROM post
ORDER BY post_created_at DESC;
```

Les données affichées dans le HTML doivent être échappées avec :

```php
htmlspecialchars($value, ENT_QUOTES, 'UTF-8');
```

---

2. create.php - Ajouter un article

Cette page contient un formulaire en méthode **POST** avec les champs suivants :

- titre (obligatoire)
- contenu (obligatoire)
- auteur (obligatoire)

À la soumission :

- vérifier que les champs ne sont pas vides (`trim`)
- insérer l'article en base avec PDO
- rediriger vers `index.php`

Requête SQL attendue :

```
INSERT INTO post (post_title, post_content, post_author)
VALUES (:title, :content, :author);
```

---

3. delete.php - Supprimer un article

Cette page reçoit un identifiant en **GET** :

```
delete.php?id=3
```

Comportement attendu :

- vérifier que l'id est un entier valide
- supprimer l'article correspondant
- rediriger vers `index.php`

Requête SQL attendue :

```
DELETE FROM post WHERE post_id = :id;
```

Sécurité minimale attendue

Pour ce TP :

- pas de concaténation SQL avec des valeurs utilisateur
- utilisation systématique de `prepare()` + `execute()`
- échappement des données affichées
- validation simple des entrées (champs non vides, id numérique)

La sécurité avancée sera abordée plus tard.

Usage autorisé de l'IA (obligatoire et encadré)

L'utilisation d'une IA (ChatGPT, Copilot, autre) est **autorisée et encouragée** pour :

- comprendre une erreur

- demander un exemple de requête SQL
- générer un squelette de code
- comprendre le fonctionnement de PDO

Fichier docs/IA.md

Ce fichier doit contenir **au minimum 3 prompts** utilisés pendant le TP.

Pour chaque prompt :

- le prompt utilisé
- la réponse de l'IA (copier-coller)
- ce que tu as compris (2 à 3 lignes)

Exemple :

```
Prompt : Explique la différence entre query() et prepare() en PDO. / Je ne
comprend comment fonctionne ... explique moi ... etc

Réponse de l'IA :
...

Ce que j'ai compris :
...
```

Livrables

- un répertoire TP_PDO_IA dépôt GitHub RedSheep-Lab contenant :
    - le projet complet
    - le fichier `README.md`
    - le fichier `docs/IA.md`
- le projet doit être fonctionnel

Critères d'évaluation

Ce qui compte réellement :

- le CRUD fonctionne (liste, ajout, suppression)
- PDO et requêtes préparées sont utilisés
- les entrées sont minimalement validées
- l'affichage est sécurisé
- le code est lisible et compréhensible
- le fichier `IA.md` montre une vraie compréhension

Remarque importante

Votre formateur restera disponible toute la journée pour vous aider n'hésitez pas à le soliciter.

```
```